

COSC264

Introduction to Computer Networks and the Internet

Reliable Data Transfer: Error Detection and Correction

Dr Barry Wu

WRC

University of Canterbury

Barry.wu@canterbury.ac.nz

Learning Objectives

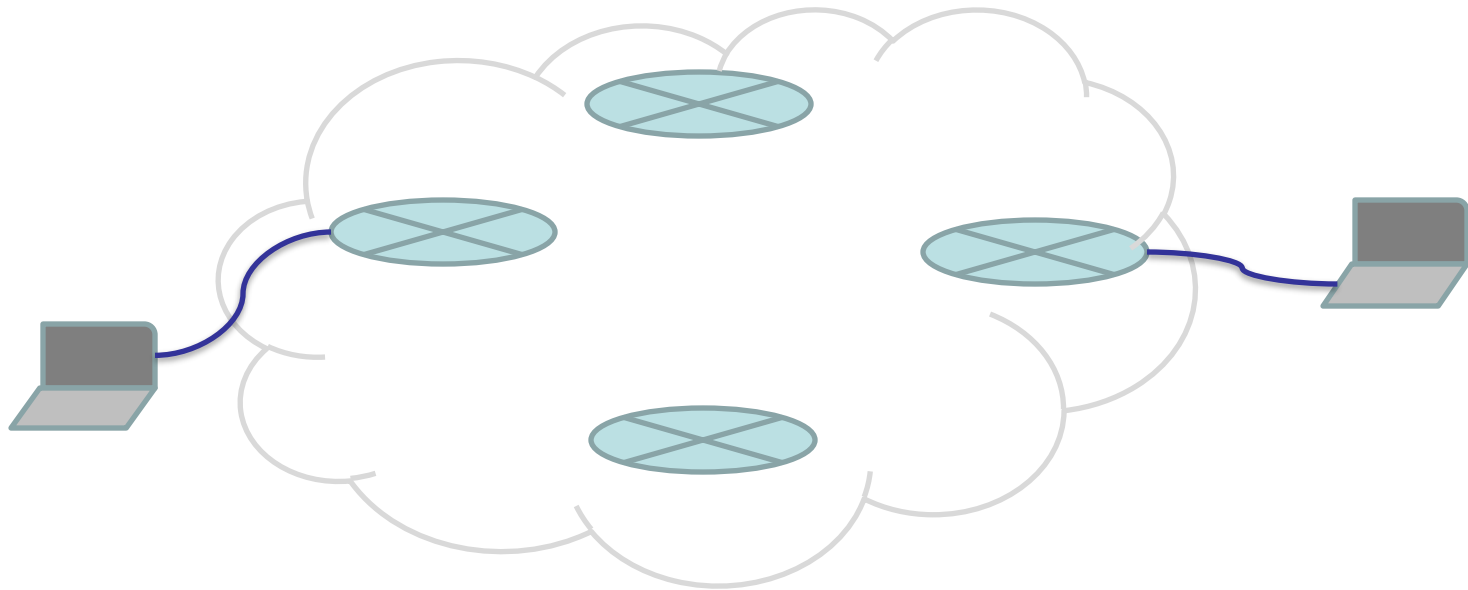
- At the end of the lectures, you will be able to:
 - Explain the basic mechanism for the use of error-detecting codes
 - Present an overview of the Internet checksum
 - Understand the operation of the cyclic redundancy check
 - Define Hamming distance
 - Explain the basic principles for forward error correction using a block code

Outline

- Error Detection
 - Parity check
 - Internet checksum
 - Cyclic Redundancy Check (CRC)
- Forward Error Correction
 - Block Code Principles

The Journey of a Packet

- Regardless of the design of the transmission system, there will be errors, such as bit errors, lost packets, and out-of-order packets.



The Journal of a Packet (cont.)

Problems	Approaches
Bit errors	Error detection and correction techniques
Lost packets	ARQ protocols
Out of order packets	ARQ protocols

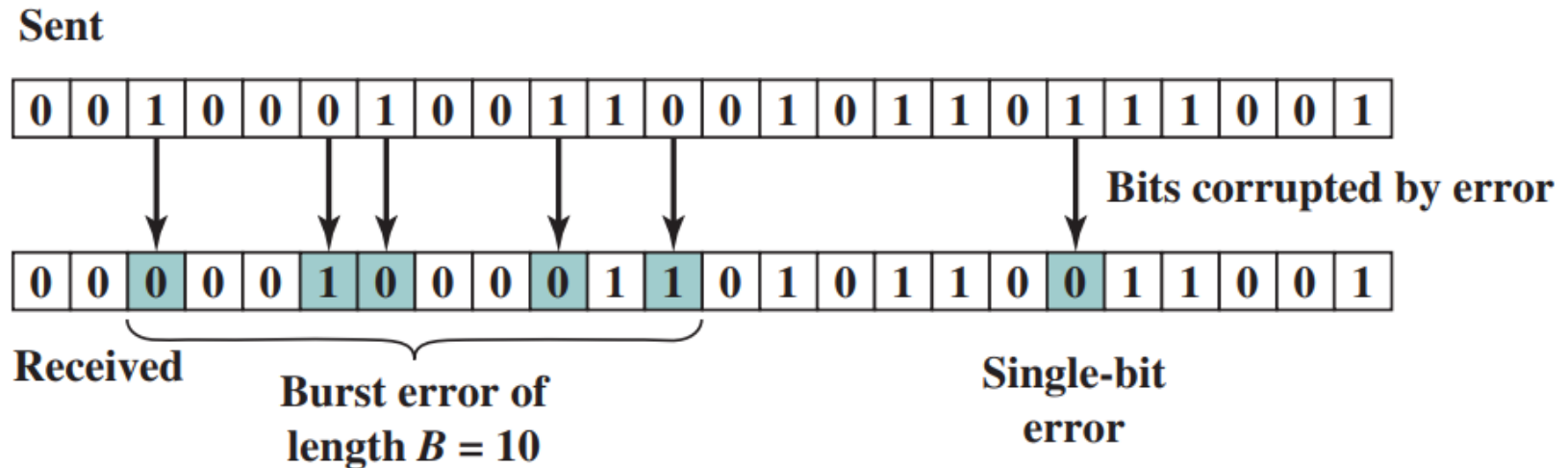
Some Causes for Transmission Errors

- Thermal noise
- Weak signal strength, interference, deliberate jamming, jitter, etc.
- A crashing router / switch / station loses all packets currently in its memory
- Packets
 - are routed in the wrong direction
 - are dropped because of congestion
 - are dropped because of insufficient resources at the receiver

Transmission errors present a design challenge for a network and a distributed application.

Types of Bit Errors

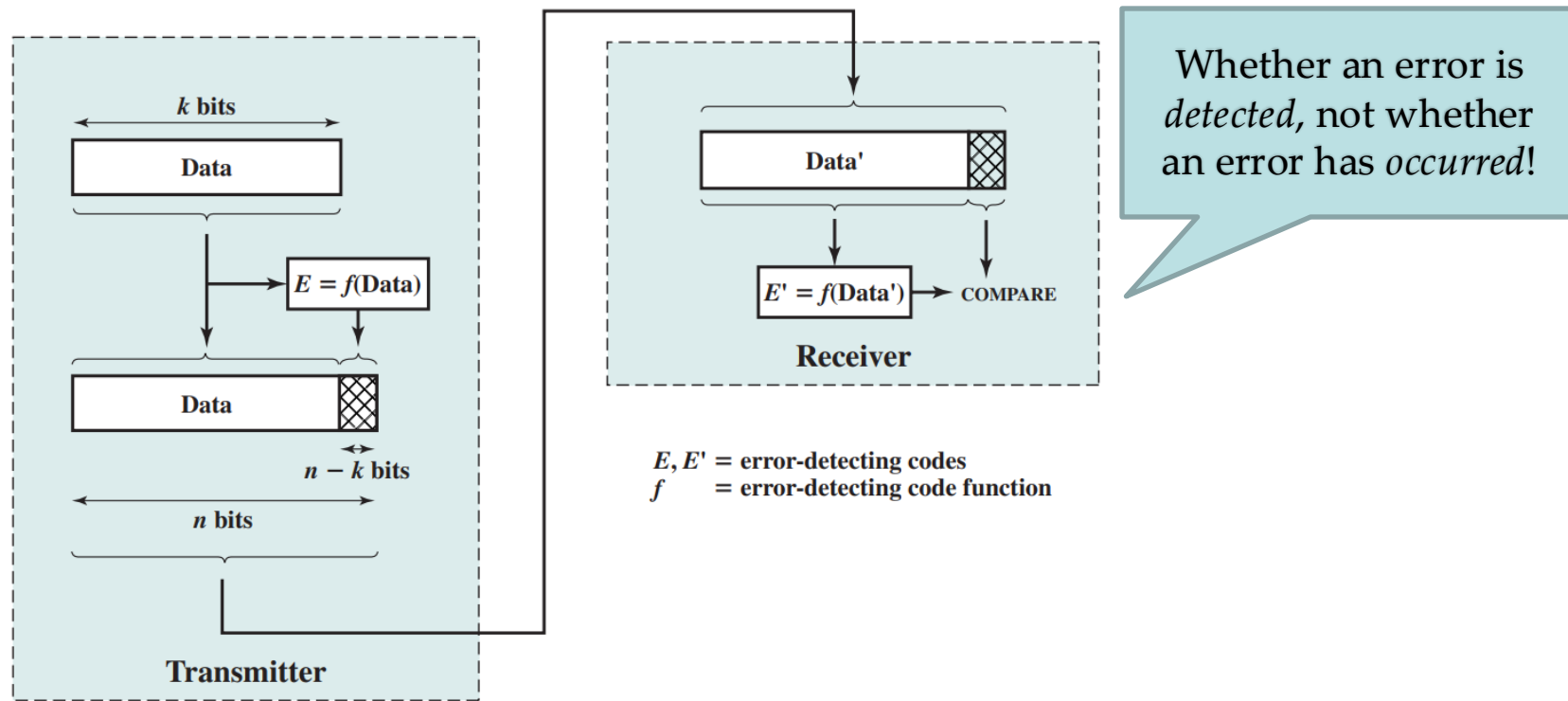
- An error occurs when a bit is altered between transmission and reception.



Error Detection

Error Detection Process

- Additional bits that constitute **an error-detecting code** are added by the transmitter.



Parity Check

- The simplest **error detecting** scheme is to append a parity bit to the end of a block of data
 - **Even parity** – use parity bit to make the number of 1's (including the parity bit) an even number.

10000101 **1**

Parity bit

10010101 **1**

Odd # of 1s --
Error detected!

- **Odd parity** – use parity bit to make the number of 1's (including the parity bit) an odd number.

10010101 **1**

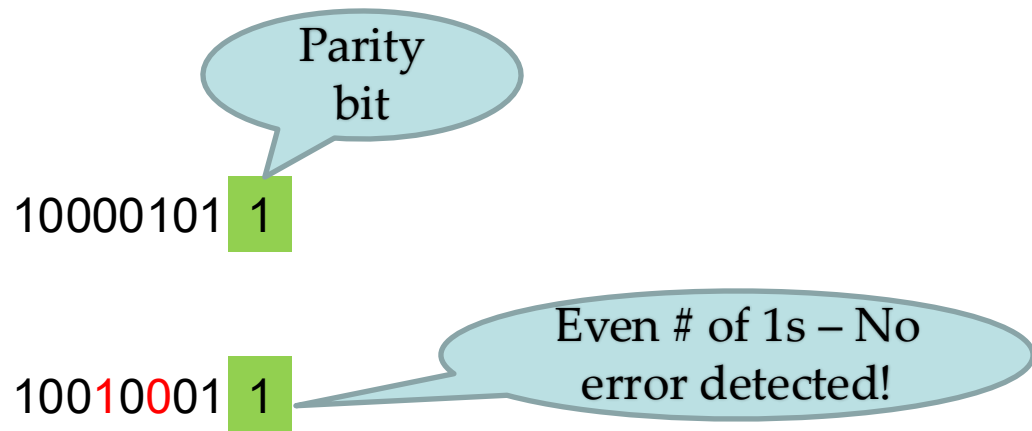
Parity bit

10000101 **1**

Even # of 1s --
Error detected!

Parity Check (cont.)

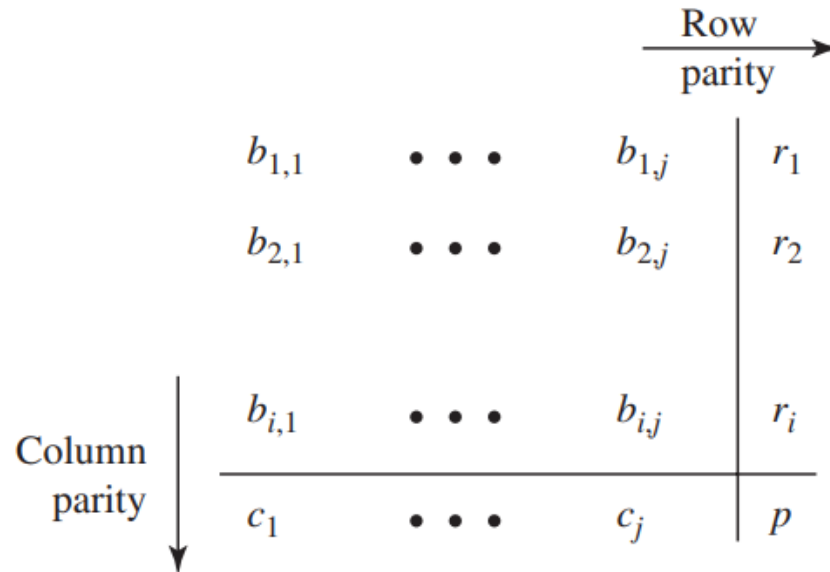
- If any even number of bits are flipped due to error, an undetected error occurs



We need a more robust error-detection scheme!

2-Dimensional Parity Check

- A parity bit r_i is appended to each row i .
- A parity bit c_j is appended to each column j .
- An overall parity bit p completes the matrix.



2-Dimensional Even Parity Example

No errors

0	1	1	1	0	1
0	1	1	1	0	1
0	1	0	0	0	1
0	1	0	1	1	1
0	0	0	1	1	0

Correctable single-bit error

0	1	1	1	0	1
0	0	1	1	0	1
0	1	0	0	0	1
0	1	0	1	1	1
0	0	0	1	1	0

Row parity error

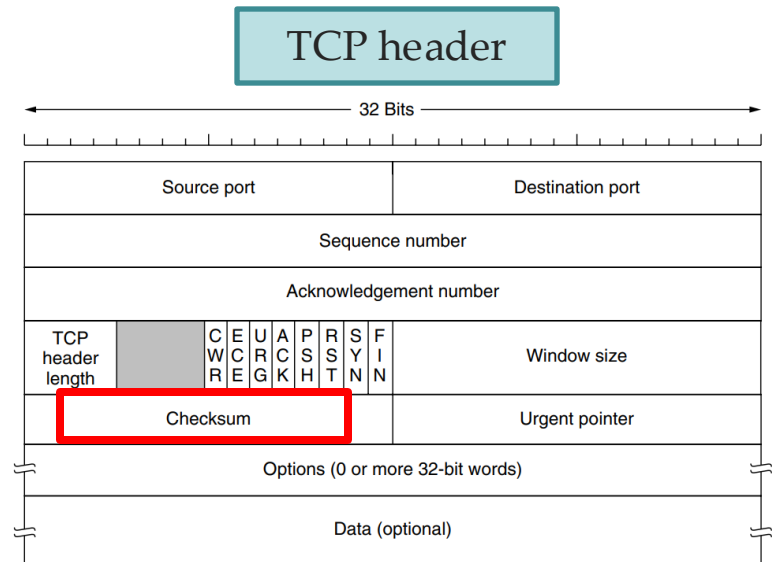
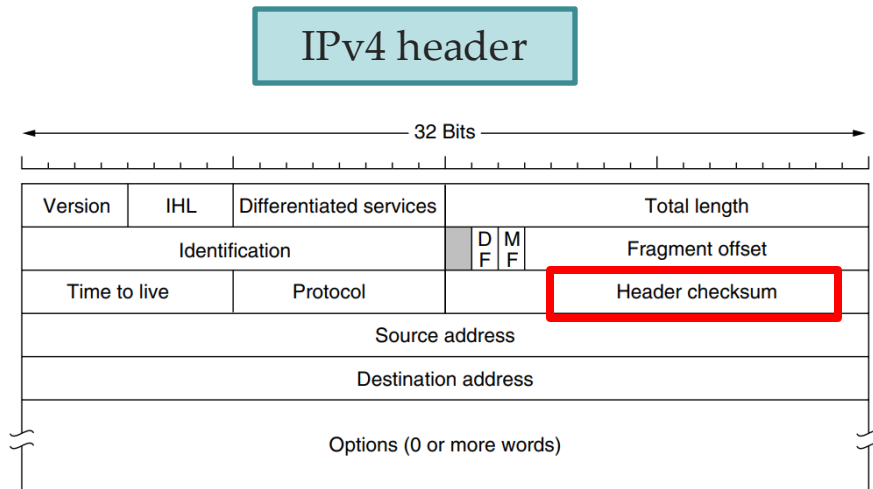
Column parity error

Undetectable error pattern

0	1	1	1	1	1	0	1
0	0	1	1	0	1	1	0
0	0	1	1	0	0	1	1
0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	0	1	1	0

Internet Checksum

- An error-detecting code
- Used in many Internet standard protocols, including IP, TCP, and UDP



Why do we need checksums at IP and TCP/UDP?

- IPv4 checksum is computed over the IP header while TCP/UDP checksum is computed over the entire segment.
- TCP/UDP and IP can belong to different protocol stacks.
 - TCP can run over a different protocol.
 - IP can carry data that will not be passed to TCP/UDP.
- TCP/UDP error detection is on an end-to-end basis.

Internet Checksum (cont.)

- The calculation makes use of:

- One's complement arithmetic (addition)

1. The two numbers are treated as unsigned binary integers and added
2. If there is a carry out of the leftmost bit, add 1 to the sum (*end-around carry*)

ex1.

$$\begin{array}{r} 0011 \\ +1100 \\ \hline 1111 \end{array}$$

ex2.

$$\begin{array}{r} 1101 \\ + 1011 \\ \hline 11000 \\ + 1 \\ \hline 1001 \end{array}$$

- One's complement operation on a set of binary digits
 - o Replace 0 digits with 1 digits and 1 digits with 0 digits

Internet Checksum (cont.)

Goal: detect errors (e.g., flipped bits) in the IP packet header

Sender to compute the checksum

- The checksum field is set to all zeros.
- The header is treated as a sequence of 16-bit integers.
- The checksum is calculated by performing the one's-complement addition of all the words in the header, and then taking the ones-complement operation of the result.
- This result is placed in the checksum field.

Receiver to verify a checksum

- The one's-complement sum is computed over the same set of octets, including the checksum field.
- The check succeeds if the result is all 1 bits.

Internet Checksum (cont.)

- Consider a header that consists of 10 octets.
 - 00 01 F2 03 F4 F5 F6 F7 00 00

Partial sum	$\begin{array}{r} 0001 \\ F203 \\ \hline F204 \end{array}$
Partial sum	$\begin{array}{r} F204 \\ F4F5 \\ \hline 1E6F9 \end{array}$
Carry	$\begin{array}{r} E6F9 \\ 1 \\ \hline E6FA \end{array}$
Partial sum	$\begin{array}{r} E6FA \\ F6F7 \\ \hline 1DDF1 \end{array}$
Carry	$\begin{array}{r} DDF1 \\ 1 \\ \hline DDF2 \end{array}$
Ones complement of the result	220D

(a) Checksum calculation by sender

hexadecimal: binary (decimal)

A:1010 (10)

B:1011 (11)

C:1100 (12)

D:1101 (13)

E:1110 (14)

F:1111 (15)

**One's complement
in hexadecimal:**

F→0

E→1

D→2

C→3

B→4

A→5

9→6

8→7

Internet Checksum (cont.)

- Consider a header that consists of 10 octets.
 - 00 01 F2 03 F4 F5 F6 F7 **22 0D**

Partial sum	0001 F203 F204
Partial sum	F204 F4F5 1E6F9
Carry	E6F9 1 E6FA
Partial sum	E6FA F6F7 1DDF1
Carry	DDF1 1 DDF2
Partial sum	DDF2 220D FFFF

(b) Checksum verification by receiver

hexadecimal: binary (decimal)

A:1010 (10)

B:1011 (11)

C:1100 (12)

D:1101 (13)

E:1110 (14)

F:1111 (15)

**One's complement
in hexadecimal:**

F→0

E→1

D→2

C→3

B→4

A→5

9→6

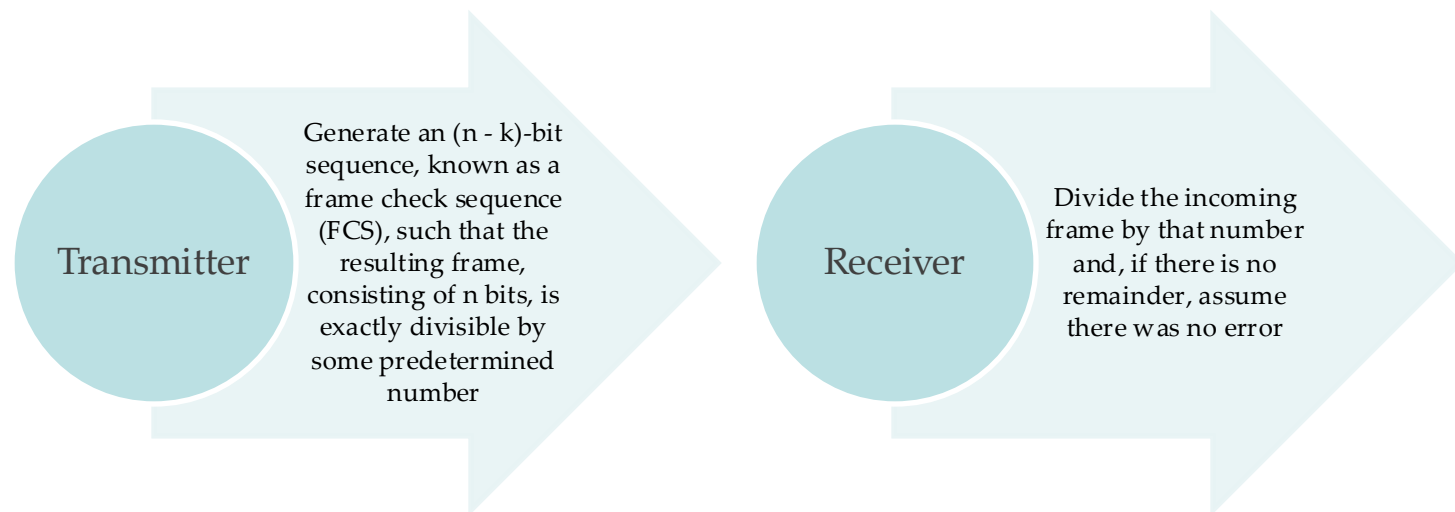
8→7

Internet Checksum (cont.)

- The primary reason for its adoption in Internet protocols is ***efficiency***.
 - Most of these protocols are implemented in software
 - The Internet checksum, involving simple operations, causes very little overhead.
- It is assumed that
 - At the lower link level, a strong error-detection code such as CRC is used.
 - The Internet checksum is an additional check for errors.
- It is considerably *less* effective than the **cyclic redundancy check (CRC)**.

Cyclic Redundancy Check (CRC)

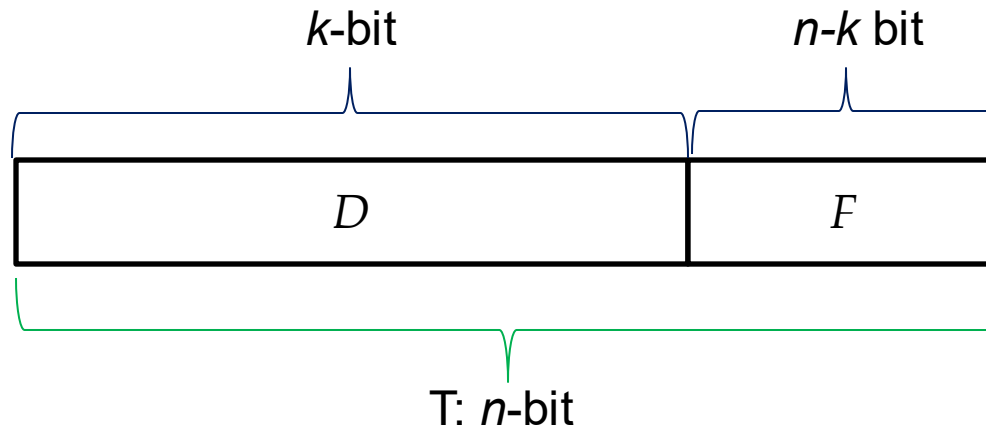
- One of the most common and powerful error-detecting codes
- Typically used in the data link layer
- Given a k -bit block of bits, or message



CRC – Notations

■ Define

- $D = k$ -bit block of data
- $F = (n-k)$ -bit FCS
- $P =$ pattern of $n-k+1$ bits (the predetermined divisor)
- $T = n$ -bit frame to be transmitted



- We would like T to be divisible by P (using *modulo-2 arithmetic*).
 - $F = \text{remainder of } (D * 2^{n-k}) / P$

CRC – Notations (cont.)

How to choose the pattern P?

- The pattern P is chosen to be one bit longer than the desired FCS.
- The exact bit pattern chosen depends on the type of errors expected.
- It is *required* that the highest-order bit = 1;
- It is *optional but common* that the lowest-order bit = 1.

CRC – Modulo-2 Arithmetic

- To calculate FCS is like doing long division in regular arithmetic, but with differences.
- Modulo-2 arithmetic
 - Addition and subtraction: use binary addition with no *carries*, just *XOR*
 - Multiplication and division: the same as in base-2 arithmetic but addition or subtraction is done without carries or borrows (*XOR*)

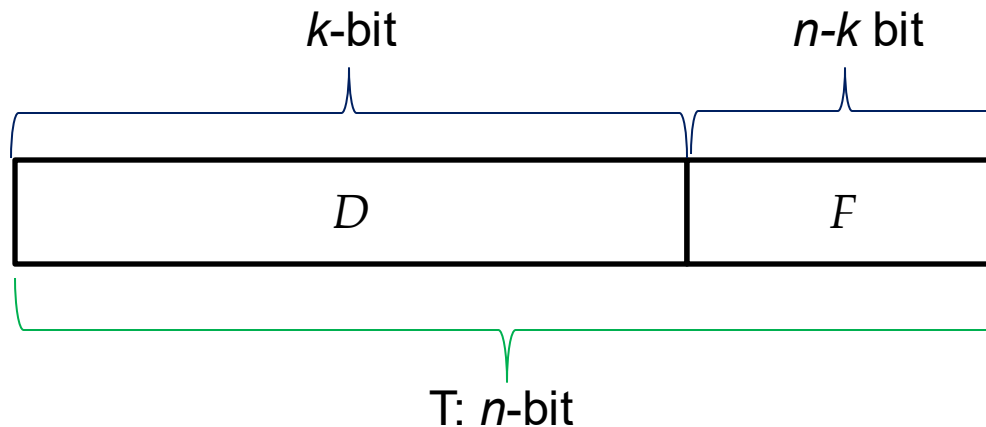
$$\begin{array}{r} 1111 \\ +1010 \\ \hline 0101 \end{array}$$

$$\begin{array}{r} 1111 \\ -0101 \\ \hline 1010 \end{array}$$

$$\begin{array}{r} 11001 \\ \times 11 \\ \hline 11001 \\ 11001 \\ \hline 101011 \end{array}$$

CRC – An Example

- Message $D = 1010001101$ (10 bits)
- Pattern $P = 110101$ (6 bits)
- FCS $F =$ to be calculated (5 bits)
- Thus, $n=15$, $k=10$, and $(n-k) = 5$



CRC – An Example (cont.)

P , as divisor

1 1 0 1 0 1

A divisor is said “to go into” a dividend if the dividend has equal number of bits as the divisor! (heading zeros not counted)

1 1 0 1 0 1 0 1 1 0

Quotient, not used

1 0 1 0 0 0 1 1 0 1 0 0 0 0 0

$D \cdot 2^{n-k}$, as dividend

1 1 0 1 0 1

1 1 1 0 1 1

1 1 0 1 0 1

1 1 1 0 1 0

1 1 0 1 0 1

1 1 1 1 1 0

1 1 0 1 0 1

1 0 1 1 0 0

1 1 0 1 0 1

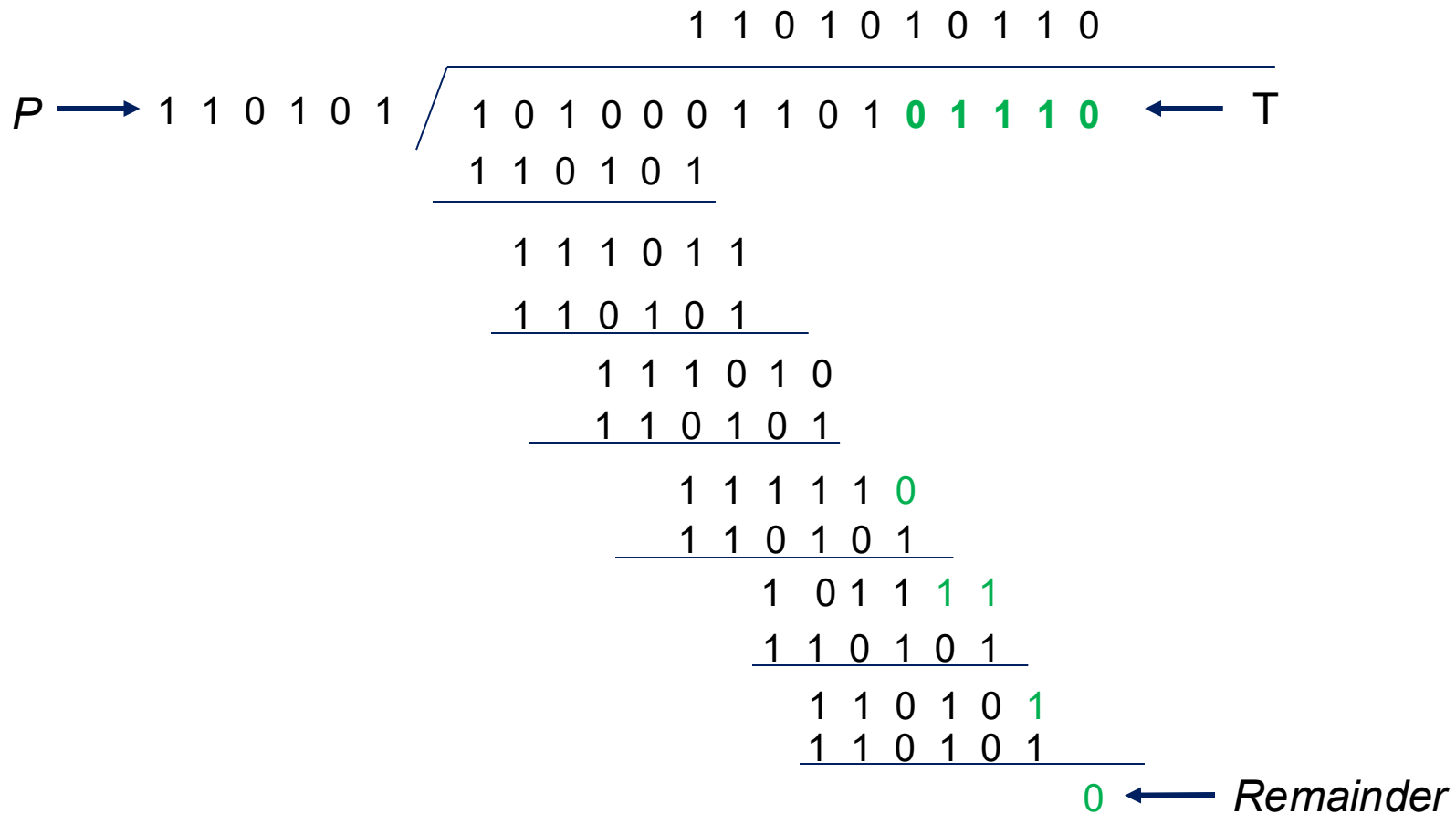
1 1 0 0 1 0

1 1 0 1 0 1

1 1 1 0 ← Remainder

- The remainder is added to $D \cdot 2^{n-k}$ to give $T = 1010001101$ 01110

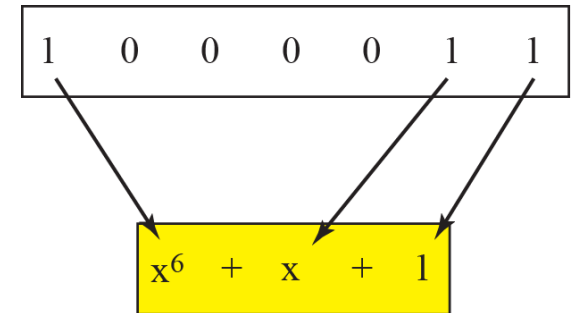
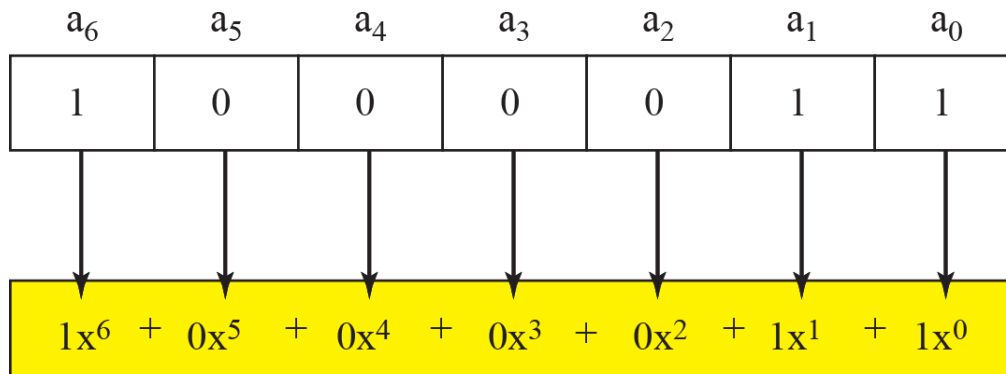
CRC – An Example (cont.)



- Because there is no remainder, it is assumed that there have been no errors.

CRC, also known as polynomial codes

- Express all values as polynomials in a dummy variable X , with binary coefficients.
 - The power of each term shows the position of the bit.
 - The coefficient shows the value of the bit.



CRC – Some Standard Polynomials

- Four versions of $P(X)$ are widely used.
 - **CRC-12**: transmissions of streams of 6-bit characters
 - **CRC-16/CRC-ANSI**: various communication protocols, including serial communication standards like Modbus
 - **CRC-CCITT**: telecommunications applications
 - **CRC-32/IEEE-802**: network protocols, such as Ethernet, ZIP file compression, PNG image format, and other file transfer protocols

$$\text{CRC-12} = X^{12} + X^{11} + X^3 + X^2 + X + 1 = (X + 1)(X^{11} + X^2 + 1)$$

$$\text{CRC-ANSI} = X^{16} + X^{15} + X^2 + 1 = (X + 1)(X^{15} + X + 1)$$

$$\begin{aligned}\text{CRC-CCITT} = X^{16} + X^{12} + X^5 + 1 &= (X + 1)(X^{15} + X^{14} + X^{13} + X^{12} \\ &+ X^4 + X^3 + X^2 + X + 1)\end{aligned}$$

$$\begin{aligned}\text{IEEE-802} = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 \\ + X^7 + X^5 + X^4 + X^2 + X + 1\end{aligned}$$

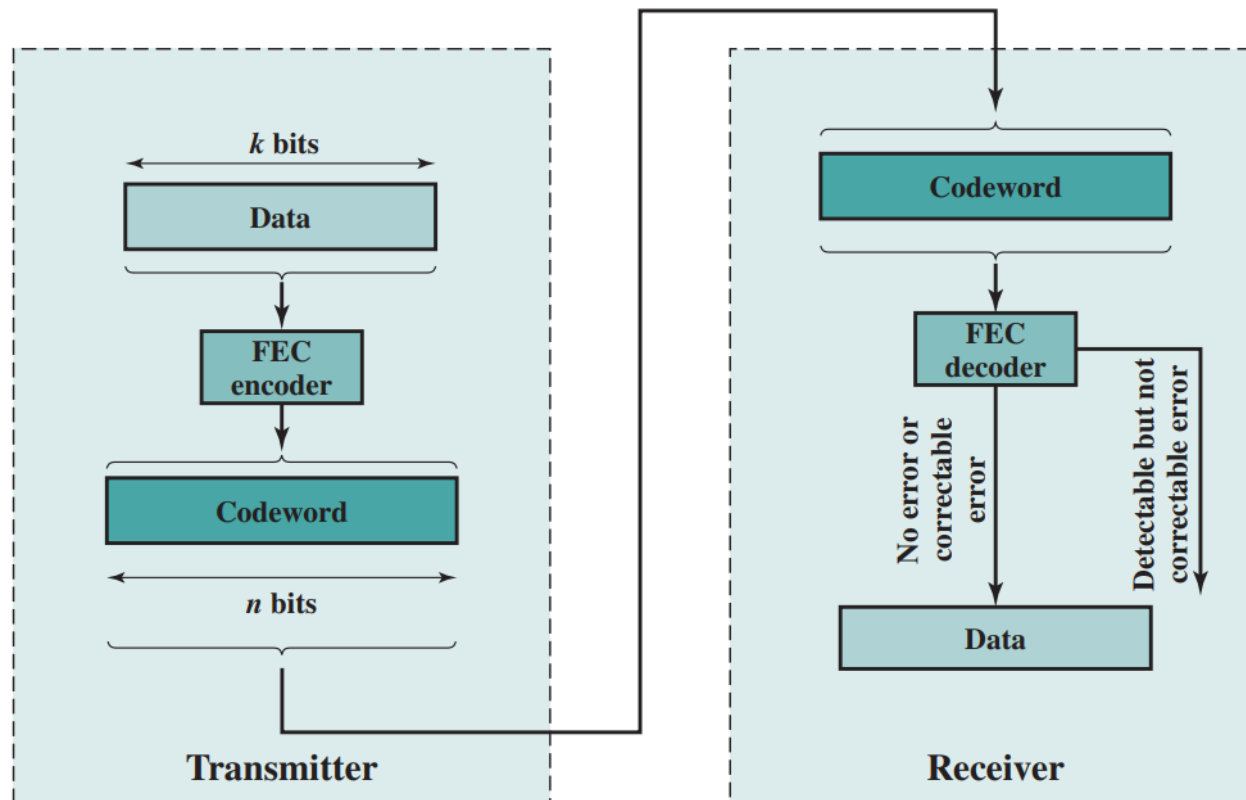
Error Correction

Forward Error Correction (FEC)

- Typically, retransmission is needed when there are bit errors
- Not good enough for **wireless applications**:
 - High bit error rate (BER) on a wireless link
 - Long propagation delay on satellite links
- **Need to correct errors on the basis of the bits received**

FEC Process

- A message is divided into blocks (*block coding*), every k bits, called a *data block*.
- FEC encoder encodes the data block; the resulting n -bit block is called a *codeword*.



An Example

- Let us assume that $k = 2$ and $n = 3$.

Data block	Codeword
00	000
01	011
10	101
11	110

Case 1 (no errors): 000 is received given data block = 00; → extract 00.

Case 2 (detectable errors): 111 is received given data block = 11; → bit error detected

Case 3 (undetectable errors): 110 is received given data block = 10;
→ extract 11, no bit error detected.

Where is the ability to correct errors?

Block Code Principles

■ Hamming distance

- $d(v_1, v_2)$ between two n -bit binary sequences v_1 and v_2 is *the number of bits in which v_1 and v_2 disagree*.
- Example: $v_1=011011$, $v_2=110001$, $d(v_1, v_2) = 3$

Block Code Principles (cont.)

- Example: for $k = 2$ and $n = 5$
- A codeword block is received with the bit pattern 00100

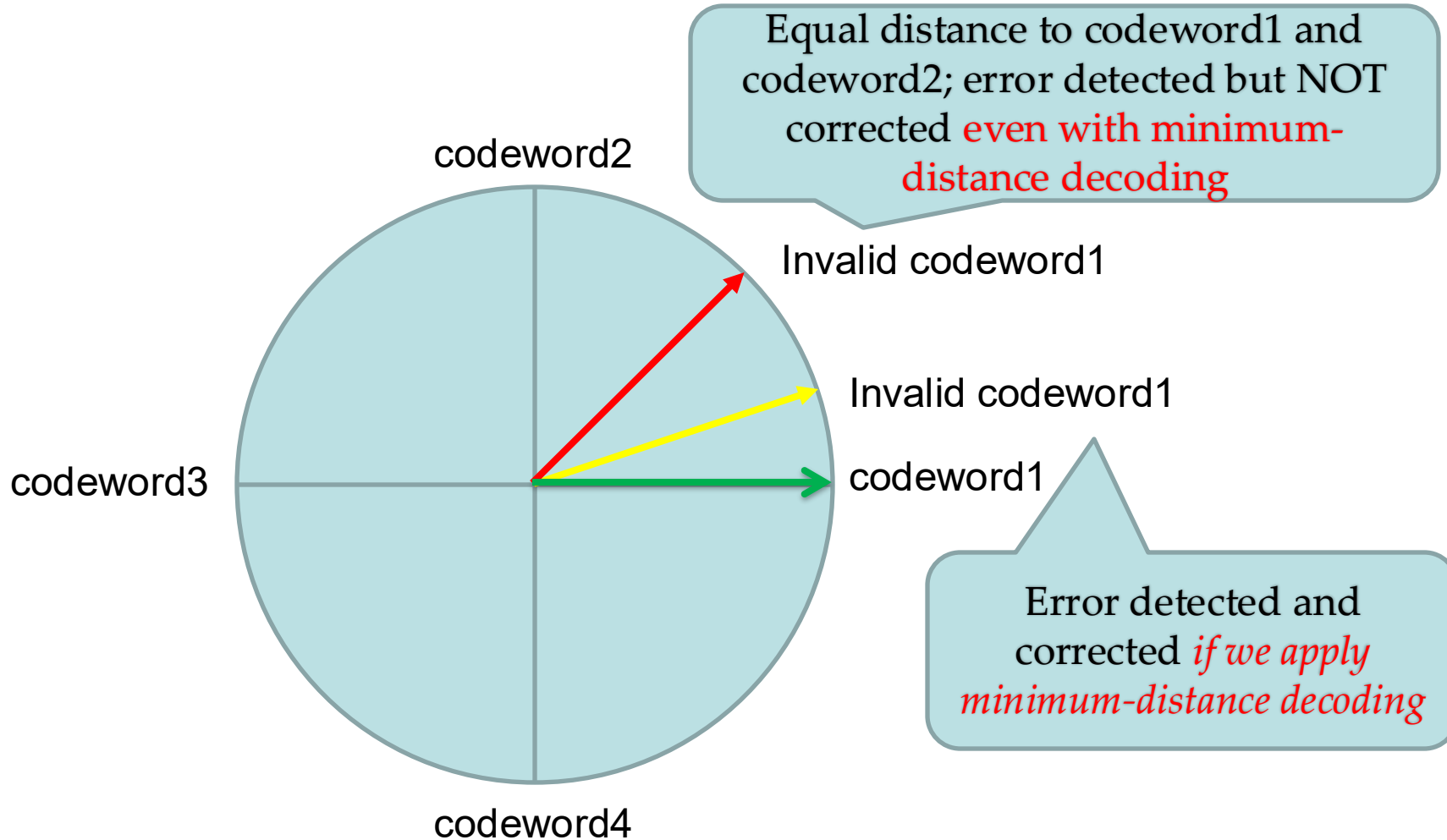
Data Block	Codeword
00	00000
01	00111
10	11001
11	11110

00100	Single bit change	Codeword 00000	Data block 00
00100	Two-bit change	00111	01
00100	Four-bit change	11001	10
00100	Three-bit change	11110	11

Block Code Principles (cont.)

- If an invalid codeword is received, the valid codeword that is closest to it (**minimum distance**) is selected.
- This will only work if there is a unique valid codeword at a minimum distance from each invalid codeword.
 - For 2-bit data blocks to be encoded into 5-bit codewords, there are $2^5 = 32$ possible codewords.
 - 4 out of these 32 codewords are valid, leaving 28 invalid codewords.

Block Code Principles (cont.)



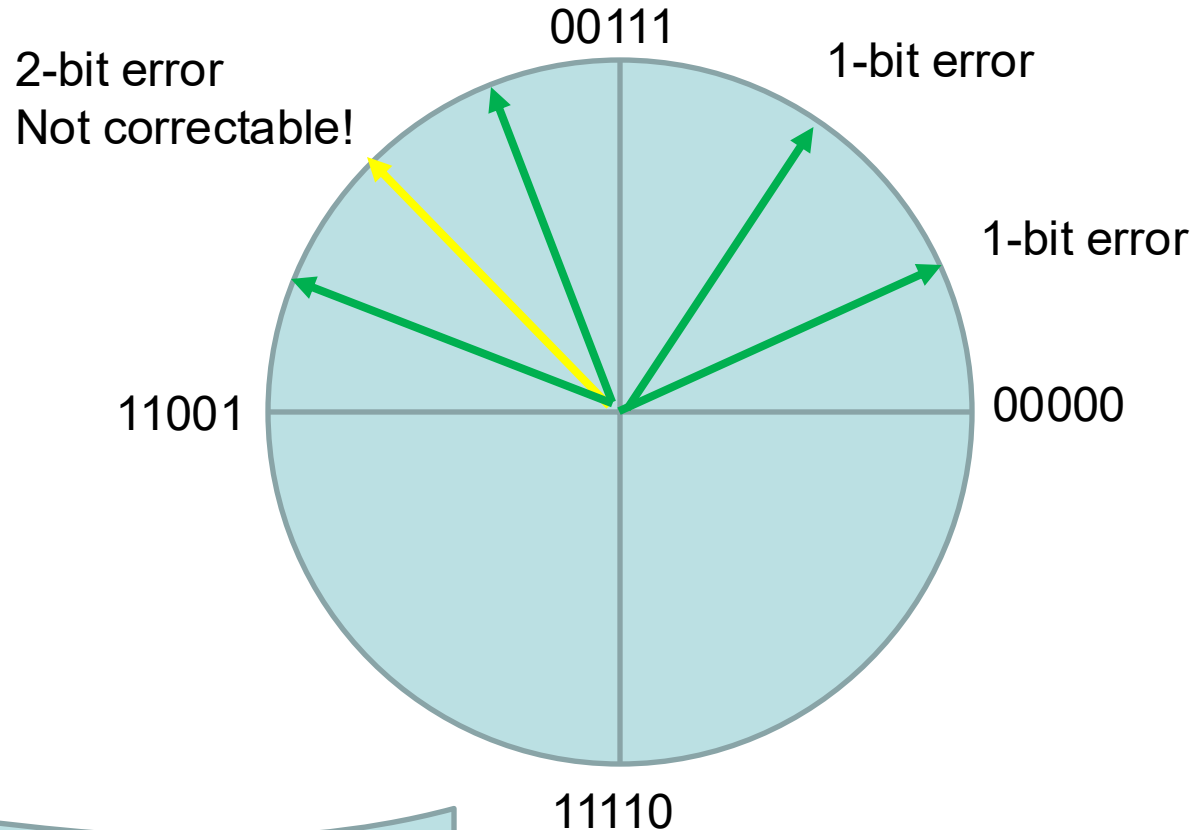
An Example of Invalid Codewords

Invalid Codeword	Minimum Distance	Valid Codeword	Invalid Codeword	Minimum Distance	Valid Codeword
00001	1	00000	10000	1	00000
00010	1	00000	10001	1	11001
00011	1	00111	10010	2	00000 or 11110
00100	1	00000	10011	2	00111 or 11001
00101	1	00111	10100	2	00111 or 11001
00110	1	00111	10101	2	00111 or 11001
01000	1	00000	10110	1	11110
01001	1	11001	10111	1	00111
01010	2	00000 or 11110	11000	1	11001
01011	2	00111 or 11001	11010	1	11110
01100	2	00000 or 11110	11011	1	11001
01101	2	00111 or 11001	11100	1	11110
01110	1	11110	11101	1	11001
01111	1	00111	11111	1	11110

Pairwise Distances between Valid Codewords

Data Block	Codeword
00	00000
01	00111
10	11001
11	11110

$d(00000, 00111) = 3$;
 $d(00000, 11001) = 3$;
 $d(00000, 11110) = 4$;
 $d(00111, 11001) = 4$;
 $d(00111, 11110) = 3$;
 $d(11001, 11110) = 3$;



The error-correction ability is related to the minimum Hamming distance of the codewords.

Summary

- CRC and FEC (the data link layer)
- IPv4 header checksum (the network layer)
- TCP/UDP checksum (the transport layer)

References

- James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, Addison Wesley, 3rd edition, 2005.
 - Chapter 5 The Link Layer and Local Area Network (5.2)
- William Stallings, Data and Computer Communications, Pearson, 10th edition, 2013.
 - Chapter 6 Error Detection and Correction
- Andrew S. Tanenbaum & David J. Wetherall, Computer Networks, Prentice Hall, 5th edition, 2010.
 - Chapter 3 The Data Link Layer (3.2)

Acknowledgements

- Lecture notes are developed based on slides from the following three sources:
 - Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2023
 - Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
 - Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005, https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm